

Colaby: A Live Collaborative Development Platform with CRDT-Based Synchronization

Harikumar M

Assistant professor

*Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India*

harikumar.m@rajalakshmi.edu.in

Pranaav kumar J

Student

*Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India*

230701233@rajalakshmi.edu.in

Raghavan P S

Student

*Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India*

230701250@rajalakshmi.edu.in

Pranav Ram S

Student

*Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India*

230701234@rajalakshmi.edu.in

Praveen Somasundaram

Student

*Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India*

230701246@rajalakshmi.edu.in

Abstract—Geographically distributed development teams often find themselves coordinating continuously in regard to project management, communication, source code development, documentation, issue tracking, and knowledge management. Typical development processes tend to separate those activities across multiple disconnected systems, resulting in context switching, fragmented state of the projects, multiple redundant channels of communication, and disruptions caused by disconnection from collaborative sessions. In this paper we introduce ColabY an end-to-end collaborative software development platform which aims at bringing the whole development process inside of the single persistent collaborative environment. ColabY integrates developer discovery, skills-based search, project creation and request, team building, project management, community, task management, documentation, issues management, collaborative whiteboarding, project notes, communication, and version control visualization inside of the collaborative development browser-based workspace on top of Eclipse Theia. Synchronization is provided using WebSocket-based event propagation in conjunction with conflict-free replicated data types (CRDTs) convergence, hybrid logical clocks, stable object identifiers, tombstone deletion management, idempotent event processing, heartbeats for presence detection, bounded retry, and event replay. The most distinctive feature of ColabY is the persistent collaborative session model, where the collaborative development environment persists beyond the scope of individual network connections, thus allowing developers to disconnect from the session and reconnect back later to continue their work in the same collaborative development workspace with state and history preserved. ColabY is implemented in terms of microservices architecture that includes API gateway, service discovery, load balancing, authentication and authorization, caching, rate limiting, PostgreSQL persistence, and centralized monitoring. All of these technologies together are aimed at reducing fragmentation of tools used throughout the software development life cycle.

Keywords—Collaborative Software Development, Persistent Sessions, Real-Time Collaboration, Conflict-Free Replicated , Web Socket, Distributed Systems

I. INTRODUCTION

For many years, software development has increasingly shifted from a locally concentrated process to one involving distributed, remote collaboration. Software development today requires participants on the project to coordinate on source code, requirements, discussion, architecture, issues, documentation, and task assignment on the multitude of Digital Applications, where each tool is individually incredibly potent in scope and feature, but where each tool represents a silo, and moving between systems in order to achieve an end result slows the development of the software. While a developer may need an IDE, a separate file repository host to store that code, a messaging client to chat with collaborators, a video conferencing system for synchronized meetings, a whiteboard tool to outline the architecture of the system under construction and yet another project management tool in order to update and track tickets, switching between systems is context-intensive and slows development by the need to not just look at the state in each application, but also make sure that the context is maintained when shifting between them; while writing code, you might remember a particular design decision from a whiteboarding session, and before getting that back you will need to locate, load, and analyze diagrams on your computer, and then maybe ask someone on the team how to apply that decision to a particular piece of code.

Past attempts to create synchronized collaborative real-time applications have taken a variety of approaches. Similar fragmentation exists in code, communications, drawing and file manipulation in many of these earlier systems, all of which have tried to integrate those components together into one synchronous environment [12]; however, the problem exists not only in unifying development artifacts into a single system but also in persistent session structure. Current real-time applications typically model collaboration based around active connections, rather than by persisting a collaboration

session in space such that regardless of the user's connection they may jump in to continue where anyone else left off. Traditional real-time collaboration treats a collaboration as primarily defined by its network session or an actively opened room. Thus, leaving that session removes access, and the collaborative context of the session requires re-establishment from base components. In contrast, ColabY treats the collaboration as context of project level-state, rather than of particular network connection.

The goal of ColabY is to provide, not just a collaborative editor but instead a end-to-end, collaborative software development environment in which a user may traverse the entire lifecycle of software development without leaving ColabY; that would be, before even writing a line of code a collaborator may search for others, establish connections, and then join/create communities and projects to request involvement from others in code, share tasks among participants, organize, document and manage their own projects, assign code-writing work, provide documentation and instructions, communicate on designs and requirements with their team, and implement code according to that design collaboratively. It does not simply provide real-time simultaneous editing but a project lifecycle based on social collaboration and task assignment.

Specifically, the work presented here addresses those issues and contains the following contributions. We provide a conceptual end-to-end development model for software collaboration integrating developer finding, team formation, and project management, real-time communication, communities, documentation, issue management, and collaborative coding into an integrated system. We have presented a durable project-workspace model which survives user session and network disconnect. We propose a real-time collaborative development architecture which leverages Eclipse Theia, a WebSocket transport, and distributed sync. We have also developed and explained a robust distributed-consistency system that leverages a CRDT system with high-level event handling, and unique logical clocks combined with stable identifiers to ensure ordering and idempotence of events including handling of lost events and eventual replays of all operations to achieve consensus across multiple client instances; the system includes code editors, whiteboards, documentation storage and issue tracking, alongside project state tracking all within a collaborative environment. Finally we describe a micro-service architecture for the system, optimized for both independent service scalability and central control access to the applications features and context.

II. RELATED WORK AND LITERATURE SURVEY

Collabode [1], by Goldman et al., is a web-based IDE that was built to showcase both simultaneous source code editing as being important and how concurrent editing causes compilation failures. However, Collabode simply targets collaborative coding, rather than the full software development process as ColabY does. ColabY puts collaborative coding into a larger project context which includes developer discovery, project management, task synchronization, communication, documentation and

permanent working spaces. There is another related work which attempts to solve this underlying problem, consistency problem of the concurrent editing. The operational transformation (OT) was originally proposed to make the real-time groupware applications consistent. Ellis et al. Proposed operational transformation that brings the concurrent operation into harmony with those already observed on any replica [2]. Later on, Shapiro et al. Presented conflict-free replicated data types

TABLE I
Comparison of collaborative development approaches

Capability	Conventional IDE	Collaborative Editor	ColabY
Source-code editing	Yes	Yes	Yes
Concurrent editing	No	Yes	Yes
Voice communication	No	Limited/ External	Yes
Shared canvas	No	No	Yes
Project management	Limited	Limited	Yes
Developer discovery	No	No	Yes
Communities	No	No	Yes
Skill-based search	No	No	Yes
Issue management	External	External	Yes
Documentation	External	External	Yes
Persistent workspace	Varies	Session-oriented	Yes
Integrated dev. lifecycle	No	Partial	Yes

(CRDTs) and showed that replicas can converge under concurrent updates without requiring synchronous coordination for every modification [3]. Kleppmann, Mulligan, Gomes, and Beresford extended CRDT theory to hierarchical, tree-structured data, presenting a move operation for replicated trees that preserves a valid, cycle-free structure under concurrent moves ColabY directly relevant to synchronizing a shared project filesystem [4]. ColabY adopts the CRDT paradigm rather than OT for **TABLE I**

CRDT synchronization is used for collaborative state synchronization, as CRDTs do not depend on a central sequence provider; OT is part of the more general synchronization design space and not part of ColabY's mechanism. CRDT synchronization is all ColabY relies on for replica state synchronization and it is the platform contribution that differs from what are discussed in Section VII (session model). ColabY, unlike previous integrated collaboration environments with a mix of synchronous application[2][4][9], is a fully unified multimodal real-time collaboration development environment where code editing, voice communication, drawing and file synchronization are blended within a single application. Similar design choices were made for event replay, tombstones and heartbeat intervals as in the mentioned research and they could serve a useful ground for the current work; it is also shown by these references that CRDT approach can manage the synchronisation of diverse media. ColabY builds on the already discussed work in that it extends the field from multimodality to the entire developing environment which includes features such as finding participants and sharing projects and communities, and, additionally and crucially, a persistency of collaborator's interaction at the session independent level. For the real-time part, there is standard WebSocket protocol provided by Fette and Melnikov for a full-duplex, persistent communication between browser and server, which does not depend on the traditional polling over HTTP. Also WebRTC specification gives standard ways to establish peer-to-peer connection and exchange different kind of data including audio between browsers and/or server [5]. ColabY extends Eclipse Theia, the first fully-functional, extensible web-based IDE [6], the architecture of which consist of two parts (client and server) communicating via WebSocket protocol and REST architecture. ColabY utilizes the existing UI and function aspects of IDE provided by Theia and builds up its unique collaboration and persistence mechanism on the existing frame, so as to providing Theia experience within the fully-functional collaborative framework on a higher level. Lewis and Fowler describe Microservice Architecture as a style of developing a single application as a suite of small services, named microservice, running in its own process and communicating, over HTTP with plain data, with a set of other services so as to building it as a complex distributed system [11]. This architecture is adopted at the backend stage of our system and will be discussed more in the next Section. In Table I we tried to represent where ColabY fits by looking at conventional IDEs, concurrent editors and multimodal collaborative systems based on various issues considered. As can be concluded from the Table above, concurrent text editing, voice communication and canvas have become common across the multiple platforms examined

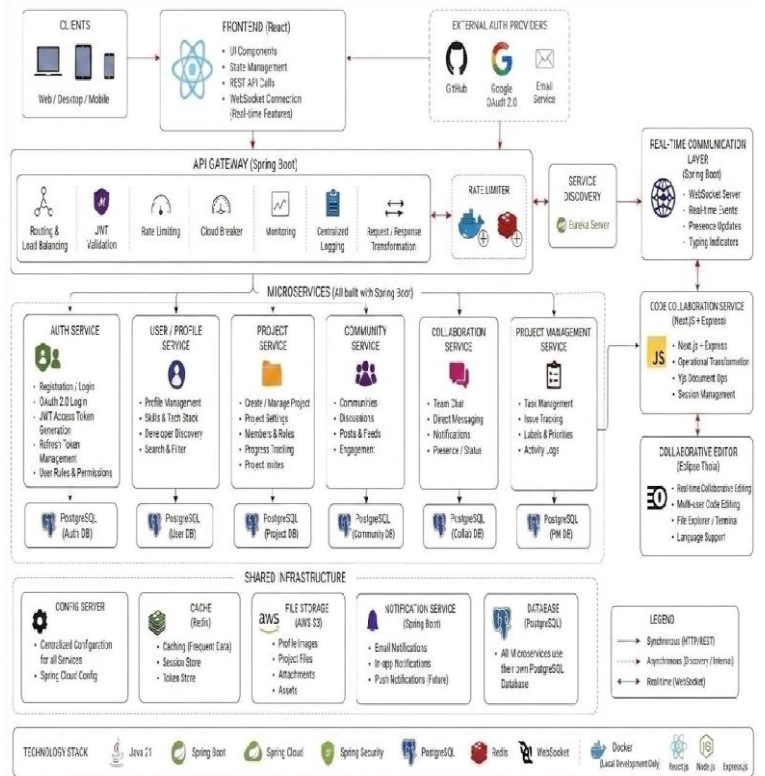
III. PROBLEM DEFINITION AND DESIGN OBJECTIVES

Existing collaboration development approaches suffer from three main issues. Firstly, fragmentation; developers need several separate applications that are unaware of each other for performing distinct tasks such as real-time communication, coding, brainstorming and issue tracking.

Secondly, disconnection; information, task, discussion, code all coexist in fragmented forms without a unified project overview. Lastly, session limitation; collaborative working is usually bound to a temporary connection or session so a user on disconnection not only interrupts the activity but rather loses its place and context. Our system, ColabY, takes a unified, project-driven approach for all types of collaboration work to fix those limitations. Guided by this problem definition, the system is designed against ten objectives: reducing context switching between tools; integrating the full software-development lifecycle within one platform; enabling concurrent real-time development; maintaining distributed state consistency across replicas; preserving project state across disconnections; supporting developer discovery and team formation; providing integrated communication supporting horizontally scalable backend services enforcing authentication and authorization; and recovering gracefully from temporary network failures. These objectives are evaluated in Section IX through functional, performance, synchronization, and persistent-session experiments.

IV. SYSTEM ARCHITECTURE

Fig. 1. Layered system architecture of ColabY.



The ColabY platform implements layered, modular architecture described and in Figure 1. A browser based interface allows clients to access development community features (developer and project discoverability) as well as a collaborative workspace including a project dashboard.

All client traffic routes to an API gateway through HTTPS connections and WebSocket based connections to enforce authorization, routing and rate limiting of backend services.

The API layer supports backend micro-services for user profiles and developers and the user profile service provides

skill discovery and support for registration, authentication, profile editing and the aggregation of a developer skill profile based on provided information as well as observed actions (such as the contributions listed) as well as community membership, task history, issue history and documentation contributions.

A searchable developer catalog is enabled based on a provided skill set-for instance, if a project needs someone with "Spring Boot, PostgreSQL, React, Docker, distributed systems", a search could filter a developer's ability to match. An independent service is responsible for persistent applications states such as the project, including its members, tasks, issues, contributions history and documentation, together with any real-time collaborative features supported in terms of common document state. The ColabY has also a non-transient application level contribution/reputation system where contributions, a set of projects completed, skills shown and community participation are tracked which can also be augmented with application specific badges for a project completed or a community question being answered. The collaborative platform supports both individual and collaborative real-time working in shared, real-time environments through a collaborative layer described further below in section VI which handles real-time updates via web sockets and consistency management. The overall service is layered.

V. END-TO-END COLLABORATIVE DEVELOPMENT WORKFLOW

To start using ColabY the most common path is to begin by finding a developer. By creating a profile displaying the necessary skills and the experience history, a developer can be found through the search function of ColabY's catalogue that indexes developers based on specific skillsets the project owners can look up. Subsequently the project owners can take the project forward, by creating the project and writing its specifications and requirements and searching for the most appropriate developers to send a collaboration request to, and compose the development team. This is a structured and planned process of developing the system, but the initial part relies on a correct searching of suitable members with matching skill sets for a certain project instead of directly jumping into writing the code. After assembling a team, the project itself can be structured in the project dashboard where every task, progress in the development, project documentation and status of each project component will be presented. Underneath it all a shared space integrates and includes all the tools for creating the project design and writing the code, from a source-code editor called Eclipse Theia that enables cooperative code writing, to a sharing whiteboard for drawing diagram schemes and exchanging ideas and a real-time voice communication and text chat tool for keeping in touch and exchanging thoughts asynchronously or synchronously. Also there's a database where project members can store documentation and decisions made regarding the technical requirements and the system design, a dashboard for visualizing problems and the

tasks and tracking their resolution with the help of a issue tracking system, and also a development progress view visualized through a version-control diagram which maps out the past contributions from all the developers working on the project. What separates the services above is how they operate on a shared space rather than on disparate data storage systems and not individually, therefore a design drawn on the whiteboard, a task initiated through a message and a problem report tied to a particular committed code change remain all consistent with the one single system state for a project.

VI. REAL-TIME COLLABORATION AND SYNCHRONIZATION ENGINE

A. Eclipse Theia as the Editing Substrate

ColabY uses Eclipse Theia as the browser-based IDE foundation for its collaborative code editor. Theia is structured as a frontend and backend that communicate over WebSockets and REST [6], and within ColabY, Theia is responsible for the editing experience ColabY syntax highlighting, language services, and the general IDE interface ColabY while ColabY's own collaboration layer is responsible for project-level synchronization and persistence around it. Theia therefore supplies the IDE experience, and ColabY supplies the collaborative architecture that keeps multiple Theia instances, and the rest of the platform, consistent with one another.

B. Event-Based Collaboration Model

Every user action inside the collaborative workspace Colab Y inserting or deleting text, creating or renaming a file, moving a cursor, sending a chat message, drawing a stroke on the whiteboard, or updating an issue or task ColabY is converted into a structured event. Conceptually, an event E can be represented as the tuple

$$E = (u, o, p, t, c, id) \quad (1)$$

where u identifies the originating user, o is the operation being performed, p is the operation's payload, t is a timestamp, c is causal metadata used for ordering, and id is a globally unique event identifier. Representing INSERT, DELETE, RENAME, CREATE_FILE, DELETE_FILE, CURSOR_MOVE, CHAT_MESSAGE, DRAW_STROKE, ISSUE_UPDATE, and TASK_UPDATE operations in this common form allows the different collaborative subsystems ColabY the editor, the whiteboard, chat, and issue tracking ColabY to share a single event-processing pipeline rather than each implementing its own synchronization logic.

C. CRDT-Based State Convergence

When two clients independently generate concurrent operations OA and OB on the same piece of shared state S , a merge function must combine them into a new state

$$S' = \text{merge}(S, OA, OB) \quad (2)$$

such that every replica that has observed the same set of operations eventually converges to the same S' , regardless of the order in which those operations were received. This convergence property is the central guarantee provided by

conflict-free replicated data types [3], and it is the mechanism ColabY relies on to keep concurrently edited documents, whiteboards, and file trees consistent across clients without requiring a central lock. Libraries such as Yjs implement this style of CRDT for shared text and structured data and automatically reconcile concurrent changes [10], and ColabY's collaborative document model follows the same convergence principle. Operational transformation, described in Section II, represents an alternative class of algorithms in which concurrent operations are transformed relative to operations a replica has already observed; ColabY treats OT as part of the broader synchronization design space, but its implemented architecture prioritizes CRDT-based convergence for collaborative state.

D. Causal Ordering: Logical and Hybrid Logical Clocks

Because network delay makes it impossible to rely on wall-clock time alone to order distributed events, ColabY uses logical clocks to establish causal ordering. Under Lamport's logical-clock scheme [8], each replica maintains a counter L that is updated according to

$$L' = \max(L_{\text{local}}, L_{\text{received}}) + 1 \quad (3)$$

which is sufficient to distinguish “event A happened before event B” from “events A and B occurred concurrently,” but does not preserve any relationship to physical time. Because approximate physical-time ordering is also useful ColabY for example, when deciding which events to retain during garbage collection, or when presenting a human-readable

event history ColabY . associates events with a hybrid logical clock (HLC) of the form

$$HLC = (pt, l) \quad (4)$$

combining a physical-time component pt with a logical counter l , following the approach described by Kulkarni et al. for consistent snapshots in globally distributed systems [9]. HLC metadata attached to each event supports deterministic event ordering during processing and replay.

E. Stable Identity for Text and Tree Structures

Collaborative text editing requires a stable way to order inserted characters so that concurrent insertions at the same position remain deterministically ordered without every client having to observe the other insertion first; conceptually, each inserted element can be identified by a position identifier $PID = (\text{siteID}, \text{counter})$, with the concrete CRDT implementation typically using a richer identifier to support efficient concatenation and splitting. Because the project workspace also contains a hierarchical filesystem of folders and files, ColabY represents file-tree operations create, delete, move, and rename ColabY as operations on a tree $T = (\text{nodes}, \text{edges}, \text{metadata})$, following the tree-CRDT approach of Kleppmann et al., which guarantees that concurrent move operations cannot introduce cycles and that all replicas converge to the same valid tree [4]. To avoid a file or folder losing its identity merely because it was renamed, each node is given a stable identifier that is independent of its current name, so that ID before = ID after across a rename, allowing a rename to be distinguished from a delete followed by the creation of an unrelated object.

F. Tombstones and Deletion Handling

Deleting an object immediately can cause problems if another replica still holds an operation referring to an earlier version of that object; ColabY therefore represents deletions with tombstones rather than immediate physical removal, so that a node that has been deleted is first marked as a tombstone rather than erased outright. This allows an operation that arrives after a deletion ColabY for example, an edit made by a disconnected client before it learns of the deletion ColabY to be interpreted correctly rather than silently discarded or treated as an error.

G. Vector Clocks and Causal Dependency

To distinguish causally related operations from concurrent ones across more than two replicas, ColabY associates operations with a vector clock $V_i = [v_1, v_2, \dots, v_n]$, where each component tracks a replica's knowledge of events originating from every other replica. For two vectors V_A and V_B , $V_A < V_B$ holds when every component of V_A is less than or equal to the corresponding component of V_B , with at least one strict inequality; this relation determines whether one operation causally depends on another or whether the two occurred concurrently and must be resolved through CRDT merge rather than simple ordering.

H. Deterministic State Application

ColabY uses the **Douglas–Peucker algorithm** to simplify whiteboard strokes by removing redundant points while preserving their overall shape. For a stroke $P = \{P_1, \dots, P_n\}$, points whose perpendicular distance from the line segment is below tolerance ϵ are removed. This reduces the number of points transmitted through WebSocket synchronization, thereby lowering network and rendering overhead while maintaining visual fidelity.

I. Idempotency and Retry

Every event is assigned a unique identifier, for example $\text{eventID} = H(\text{userID}, \text{sessionID}, \text{sequence})$, so that if the same event is delivered twice . ColabY which can happen whenever a client retries a request that actually succeeded ColabY the duplicate can be detected and discarded rather than applied a second time. Complementing this, a failed event delivery is retried with a bounded exponential backoff,

$$\text{retry}_i = \min(\text{retry}_{\text{max}}, \text{base} \times 2^i) \quad (5)$$

optionally with jitter added to avoid many clients retrying in a synchronized burst, so that transient packet loss does not by itself cause state to diverge between replicas.

J. Presence, Replay, and Backlog Synchronization

Presence is maintained through periodic heartbeats of the form $HB = (\text{userID}, \text{state}, \text{lastActive}, \text{HLC})$, which let the platform distinguish an actively connected participant from one who has disconnected or gone idle. This mechanism connects directly to ColabY's persistent-session design: while a participant is disconnected, other members of the project may continue generating events, which accumulate in a durable event log rather than being lost. When the disconnected participant reconnects, the client authenticates, identifies the last event it has

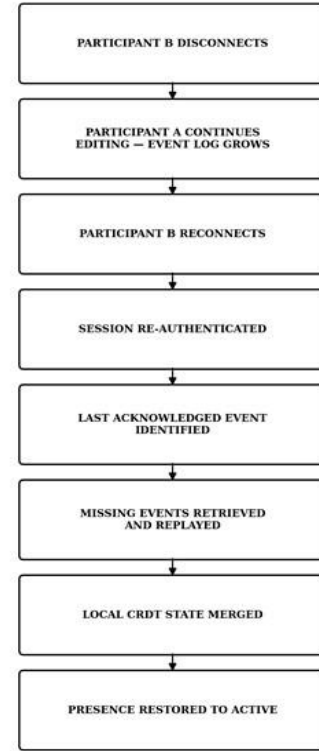
acknowledged, and the server computes the missing backlog and replays it to the client; the client then merges the replayed operations into its local CRDT state and restores its workspace and presence status. Because the reducer described in Section VI-H is deterministic and every event carries a stable identifier, HLC timestamp, and causal metadata, this replay procedure reconstructs exactly the state the client would have reached had it remained continuously connected. This event-replay mechanism is one of the strongest technical supports for the persistent-session architecture described in the next section.

VII. PERSISTENT SESSION ARCHITECTURE

Traditional real time collaborative systems might be described as a chain User session Connection so that a given user is strongly tied to a given connection, and that given connection determines the existence of a given session ColabY(intuitively: User is tied to connection, connection is tied to session) so the life of the session depends directly on that of the connection, that is, disconnecting a single user makes the session expire with it, so the collaborative context depends only on the availability of any participant. ColabY associates persisting state with the project, rather than connection. The relation is Persistent Workspace Project so when a connection to a given project is closed, we get: Persistent Workspace Project and other members can continue working on the project with out going down, and may re-join at any time, without worrying about losing any work

done between the members. A ColabY persistent workspace holds in an implicit way the project data, team members, tree of files, state of CRDT, event log, tasks, issues, notes, documentation, messages exchanged by members of the project, version and presence meta data. A participant can then join and work, then disconnected, and the project continue working without the participants until some days later, at what time the participant wants to come back, they will just need to be re-connected exactly in the place the project state has gotten to, without losing their own current working state. Figure. 2 describe how a member that was disconnected and comes back: re-authenticate using existing credential; the application discover that we want to access the persistent workspace related to our last working project; the client send his last seen version of the state and server computes difference and replayed missing operation using the causal ordering and merge it to local CRDT; the new presence value is then updated.

Fig. 2. Persistent session recovery workflow.



Because persistence in this architecture applies specifically to collaborative workspace state, ColabY deliberately does not extend the same persistence to authentication: a returning participant's authorization is still validated on reconnection, so that resuming a workspace never implies an indefinitely valid session token. This distinction ColabY persistence of collaborative state without persistence of authorization ColabY is discussed further in Section VIII.

VIII. MICROSERVICE ARCHITECTURE, INFRASTRUCTURE, AND SECURITY

With separate authentication/authorization management of their identities, ColabY's backend is composed of a set of loosely-coupled micro-services where each one of them is assigned to handle one business capability [11]. The services (authentication/ authorization, profile, social, community, project, workspace, and collaboration) are all written in spring boot within one Maven monorepo. Having a well defined range to the responsibility of each one enables independent service evolution with common centralization to authentication/authorization enforcement and routing.

Each client service call will pass over a central API gateway that authenticates and authorizes clients and routes client requests to the suitable backend services. Services are not coupled by knowing physical addresses of other services and have no idea about how and to whom should they forward their request; therefore, to solve this problem ColabY uses service discovery to locate healthy service instances, load

balanced to spread request traffic through these instances to provide higher availability and to minimize bottlenecks [12]. Data in ColabY that do not change often or are very frequently required to speed up response time, are data like; user's profile, user profiles lookup, project meta-data, content that belongs to the community, are being cached, whereas rate limiting is utilized to ensure some requests per second will pass a given endpoint to minimize service performance degradation, which are; authentication and Authorization, search profile(user's profile lookup), project-request, community and real-time events ingestion. Since ColabY has distributed services that are dependent one another on some calls; it uses a circuit breaker in front of downstream calls; thus; the failure in one of the downstream services would degrade gracefully the remaining ColabY services rather than affecting more system failures [13].

Table II
Different services and their core responsibility

Service	Primary Responsibility
Authentication Service	Registration, login, JWT, authorization
User/Profile Service	Profiles, skills, developer search
Project Service	Project creation, membership, project lifecycle
Community Service	Communities, posts, discussions
Collaboration Service	Chat, messaging, presence, real-time events
Project Management Service	Tasks, issues, progress, activity
Workspace Service	Collaborative development workspace

Authentication is performed via JSON Web Tokens; refresh tokens, instead of trusted forever, are rotated and stored in the server-side backend in Redis. Role-based authorization scopes this validation to only allow role-specific action on that particular resource, e.g., a project owner may do anything on its project, project members will only be allowed to do developer action on it, while community members are only allowed community-specific actions; the caller's real role will be checked against for any incoming requests instead of trust client's statement. The channel names are authenticated and

carried over TLS; database mutations are all checked on the server, preventing any clients from directly modifying objects that it isn't entitled to. Persistence, in ColabY, does not imply trust, as in Section VII, if one logs in to a session that had been disconnected but not properly logged out, the access rights of the participant must still be checked.

IX. EVALUATION METHODOLOGY

The implemented ColabY platform was evaluated for real-time responsiveness, scalability, collaborative consistency, and persistent-session recovery. Experiments were conducted with increasing numbers of concurrent users to measure synchronization latency, throughput, resource utilization, and recovery performance. Real-time synchronization latency was measured from event generation to successful application at the receiving client. Code edits, file operations, whiteboard events, chat messages, and presence updates were evaluated under 2, 4, 8, and 16 concurrent users.

TABLE III
Real - time performance and scalability

Users	Avg. Latency (ms)	P95 Latency (ms)	Throughput (events)
2	31	48	118
4	34	53	225
8	39	61	417
16	47	76	762

These results demonstrate a mild latency increase with an increase in the number of collaborators while still having the system propagate events quickly. The concurrent editing benchmark used overlapping operations on multiple clients and for the persistent-session recovery benchmark, a participant was disconnected and the time to restore the workspace was measured.

TABLE IV
Convergence and persistent-session recovery

Users		Operations	Convergence (%)	Offline Events	Recovery Time (ms)	Recovery (%)
2		100	100	10	142	100
4		250	100	50	218	100

8		500	100	100	361	100
16		1000	98	250	684	98

The results demonstrate consistent replica convergence and successful recovery of interrupted sessions. Recovery time increases with the number of events accumulated during disconnection, as the system must replay a larger event backlog before restoring the latest workspace state.

X. LIMITATIONS AND FUTURE WORK

Our current design is tailored to the software development collaboration aspect, and attempts to not displace all of the specific tools from every phase of the broader software development life-cycle. The nature of real time synchronization places constraints on network resources, storage capacity and processing power which escalate with increased numbers of active collaborators. CRDT state and the event log, as discussed, also grow over time necessitating snapshots and garbage collection processes to remain within limits of both storage and the replay operation. As persistent workspaces remain throughout the tool lifecycle access control management and its lifetime are of particular concern lest old state remain and potentially find its way to users without current authorization to the workspace. Future improvements include automated workspace snapshots, smart event batching for minimized network traffic during heavy concurrency and at potentially lower concurrency; offline First editing for development over poor networks; more scaleable CRDT storage solutions and project suggestion algorithms, built on the components introduced in IV, for discovery of, and skill matches with, collaborators once the core platform is solidified.

XI. CONCLUSION

ColabY aims to support end-to-end collaborative software development where developer discovery, project initiation, project management, communication, documentation, real-time design, and in real-time implementation are all integrated within a single, unified, project-centric workspace. Instead of the system merely being based on in-sync code editing and multimodal communication, ColabY covers the entire development lifecycle, integrating Eclipse Theia, WebSocket-based transport, CRDT-based synchronization, hybrid logical clocks, idempotent event processing, presence mechanisms, retries, and event replay for distributed collaborative development. Key architectural innovation includes the introduction of a notion of persistently bound collaborative sessions where instead of the lifespan of a collaborative session is bounded to real-time activities and that instead it is tied to the persistent state of the project space, in that the collaborating parties can leave and rejoin at any time via re-synchronization and backlog restoration. This ties

up the link between real-time activities and conventional, ongoing, projects while a number of microservices separate key functional components thereby improving extensibility, fault Tolerance, maintainability and security. Evaluation will investigate measurable properties such as synchronization latency, the order of convergence, time it takes to restore a session, performance when a large number of collaborators present in the system, and task velocity against conventional distributed tool sets.

REFERENCES

- [1] M. Goldman, G. Little, and R. C. Miller, "Real-time collaborative coding in a web IDE," in Proc. 24th ACM Symp. User Interface Software and Technology (UIST), 2011, pp. 155–164.
- [2] C. A. Ellis and S. J. Gibbs, "Concurrency control in groupware systems," in Proc. ACM SIGMOD Int. Conf. Management of Data, 1989, pp. 399–407.
- [3] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-free replicated data types," in Stabilization, Safety, and Security of Distributed Systems (SSS), 2011, pp. 386–400.
- [4] M. Kleppmann, D. P. Mulligan, V. B. F. Gomes, and A. R. Beresford, "A highly-available move operation for replicated trees," IEEE Trans. Parallel Distrib. Syst., vol. 33, no. 7, pp. 1711–1724, Oct. 2021.
- [5] World Wide Web Consortium, "WebRTC: Real-Time Communication in Browsers," W3C Recommendation.
- [6] Eclipse Foundation, "Theia Architecture Overview," Eclipse Theia Documentation.
- [7] I. Fette and A. Melnikov, "The WebSocket Protocol," RFC 6455, IETF, Dec. 2011.
- [8] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," Commun. ACM, vol. 21, no. 7, pp. 558–565, Jul. 1978.
- [9] S. S. Kulkarni, M. Demirbas, D. Madeppa, B. Avva, and M. Leone, "Logical physical clocks and consistent snapshots in globally distributed databases," Univ. at Buffalo, Tech. Rep., 2014.
- [10] Yjs, "Yjs: A CRDT framework for building collaborative applications," documentation.
- [11] J. Lewis and M. Fowler, "Microservices," martinowler.com, 2014.
- [12] S. Steffi Nivedita, P. U. Harsha, P. Rahul, P. Surya Tej, and P. Venkat Balaji Naidu, "A unified multi-modal real-time collaborative development environment integrating code editing, voice communication, drawing, and file synchronization," Int. J. Sci. Res. Technol., vol. 2, no. 12, pp. 385–390, 2025.